In [55]:
```
## EXP1 Image-Handling-and-Pixel-Transformations-Using-OpenCV
## NAME:- LITYA M
## REGISTER NUMBER:-212225230152
```

Step1: Load an image from your local directory and display it.

In [2]:
```
%pip install matplotlib
```

```
Requirement already satisfied: matplotlib in .\anaconda3\envs\opencv-env\Lib\site-pa
ckages (3.11.1)
Requirement already satisfied: contourpy>=1.0.1 in .\anaconda3\envs\opencv-env\Lib\s
ite-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\anaconda3\envs\opencv-env\Lib\site-
packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in .\anaconda3\envs\opencv-env\Lib
\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\anaconda3\envs\opencv-env\Lib
\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in .\anaconda3\envs\opencv-env\Lib\site-p
ackages (from matplotlib) (2.5.1)
Requirement already satisfied: packaging>=20.0 in .\anaconda3\envs\opencv-env\Lib\si
te-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in .\anaconda3\envs\opencv-env\Lib\site-pac
kages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in .\anaconda3\envs\opencv-env\Lib\site-
packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in .\anaconda3\envs\opencv-env\L
ib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in .\anaconda3\envs\opencv-env\Lib\site-pack
ages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [3]:
```
import cv2
import matplotlib.pyplot as plt
```

In [4]:
```
# Read the image using OpenCV
img = cv2.imread('dolphin image 1.jpg', cv2.IMREAD_COLOR)
```

In [5]:
```
# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

In [6]:
```
# Display the image using Matplotlib
plt.imshow(img_rgb, cmap='viridis')  # You can change 'viridis' to another cmap or
plt.title("Original Image")
plt.axis('off')  # Removes axis ticks and labels
plt.show()
```

Step2: o Draw a line from the top-left to the bottom-right of the image. o Draw a circle at the center of the image. o Draw a rectangle around a specific region of interest in the image. o Add the text "OpenCV Drawing" at the top-left corner of the image.Draw a line from the top-left to the bottom-right of the image

In [7]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')
```

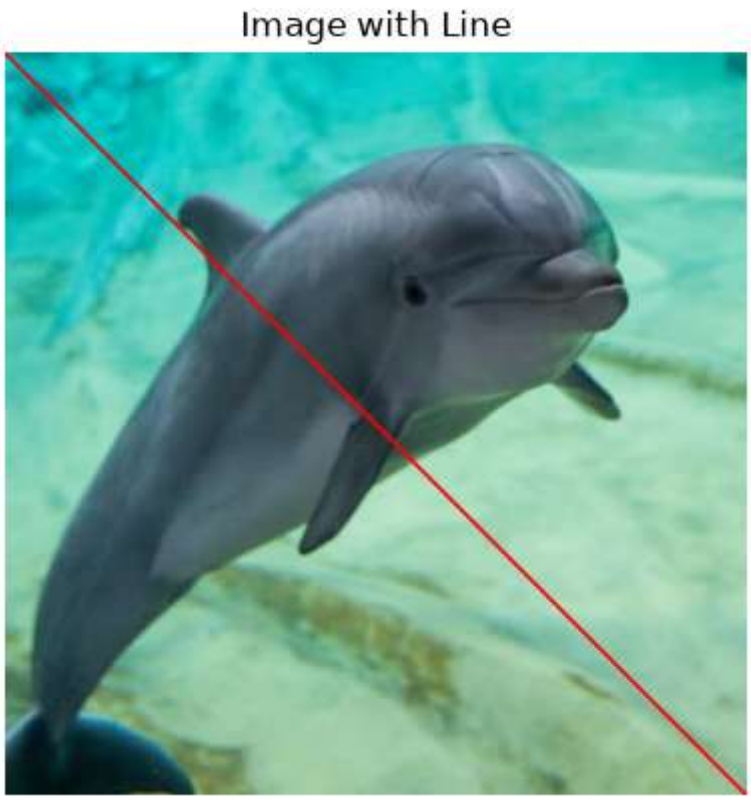
In [8]:
```
# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

In [9]:
```
img_rgb.shape
```

Out[9]:
```
(750, 750, 3)
```

In [10]:
```
# Draw a line from top-left to bottom-right
line_img = cv2.line(img_rgb, (0, 0), (750, 750), (400, 0, 0), 2) # cv2.line(image,
```

In [11]:
```
plt.imshow(line_img, cmap='viridis')
plt.title("Image with Line")
plt.axis('off')
plt.show()
```



Draw a circle at the center of the image.

In [12]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')

# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

In [13]:
```
img_rgb.shape
```

Out[13]:
```
(750, 750, 3)
```

In [14]:
```
circle_img = cv2.circle(img_rgb,(395,400),275,(255,0,0),3) # cv2.circle(image, cent
```

In [15]:
```
plt.imshow(circle_img, cmap='viridis')
plt.title("Image with Circle")
plt.axis('off')
plt.show()
```

Draw a rectangle around the whole image

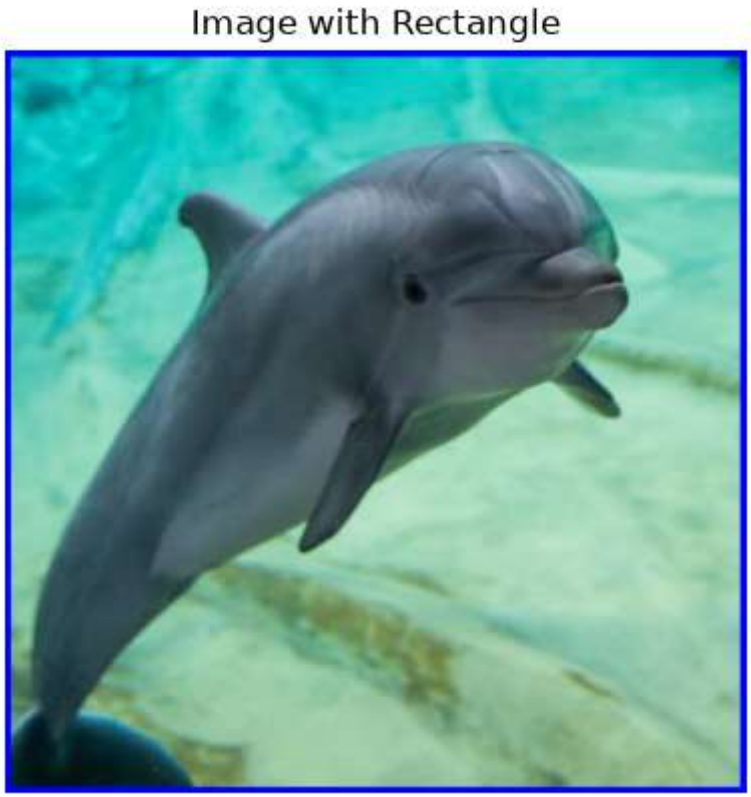
In [16]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')

# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

In [17]:
```
img.shape
```

Out[17]:
```
(750, 750, 3)
```

In [18]:
```
# Draw a rectangle around the Whole image
rectangle_img = cv2.rectangle(img_rgb, (0, 0), (749, 749), (0, 0, 255), 10)  # cv2.
```

In [19]:
```
plt.imshow(rectangle_img, cmap='viridis')
plt.title("Image with Rectangle")
plt.axis('off')
plt.show()
```



Add the text "OpenCV Drawing" at the top-left corner of the image.

In [20]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')

# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

In [21]:
```
# Add text to the image
text_img = cv2.putText(img_rgb, "Hello Dolphin", (10, 30), cv2.FONT_HERSHEY_SIMPLEX
```

In [22]:
```
plt.imshow(text_img, cmap='viridis')
plt.title("Image with Text")
plt.axis('off')
plt.show()
```

Step3: o Convert the image from RGB to HSV and display it. o Convert the image from RGB to GRAY and display it. o Convert the image from RGB to YCrCb and display it. o Convert the HSV image back to RGB and display it.

In [23]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')
```

In [24]:
```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

In [25]:
```
# Original RGB Image
plt.imshow(image_rgb)
plt.title("Original RGB Image")
plt.axis("off")
```

Out[25]:
```
(np.float64(-0.5), np.float64(749.5), np.float64(749.5), np.float64(-0.5))
```

In [26]:
```
# Convert RGB to HSV
image_hsv = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2HSV)
```

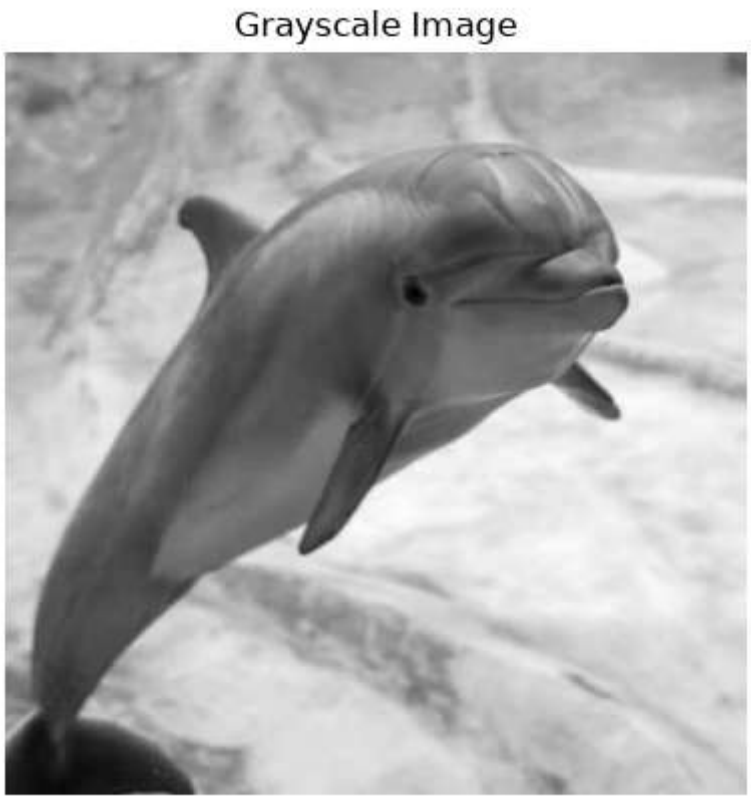
In [27]:
```
# HSV Image
plt.imshow(image_hsv)
plt.title("HSV Image")
plt.axis("off")
```

Out[27]:
```
(np.float64(-0.5), np.float64(749.5), np.float64(749.5), np.float64(-0.5))
```

In [28]:
```
# Convert RGB to GRAY
image_gray = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2GRAY)
```

In [29]:
```
# Grayscale Image
plt.imshow(image_gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")
```

Out[29]:
```
(np.float64(-0.5), np.float64(749.5), np.float64(749.5), np.float64(-0.5))
```



In [30]:
```
# Convert RGB to YCrCb
image_ycrcb = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2YCrCb)
```

In [31]:
```
# YCrCb Image
plt.imshow(image_ycrcb)
plt.title("YCrCb Image")
plt.axis("off")
```

Out[31]:
```
(np.float64(-0.5), np.float64(749.5), np.float64(749.5), np.float64(-0.5))
```

In [32]:
```
# Convert HSV back to RGB
image_hsv_to_rgb = cv2.cvtColor(image_hsv, cv2.COLOR_HSV2RGB)
```

In [33]:
```
plt.imshow(image_hsv_to_rgb)
plt.title("HSV to RGB Image")
plt.axis("off")
```

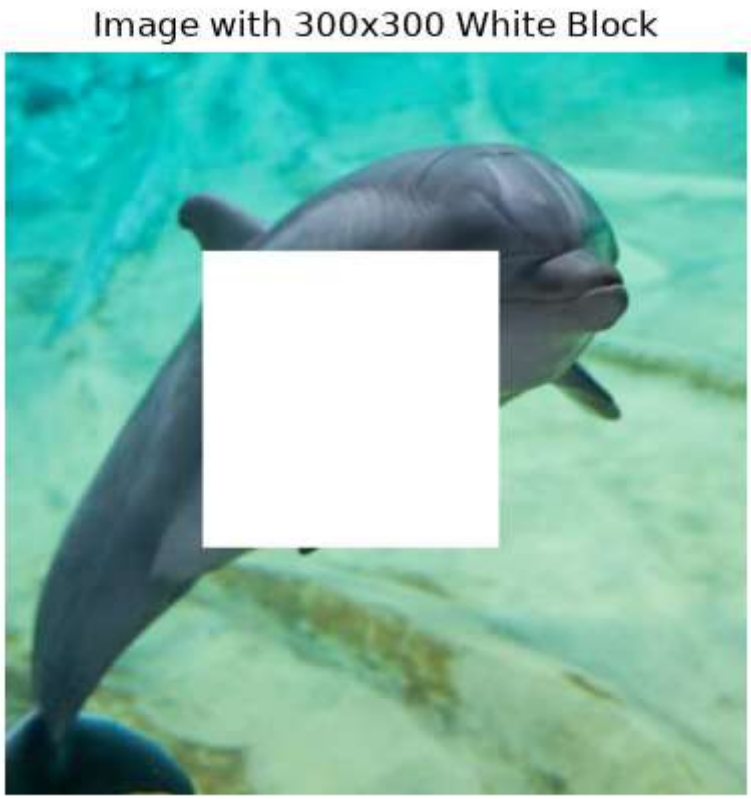
Out[33]:
```
(np.float64(-0.5), np.float64(749.5), np.float64(749.5), np.float64(-0.5))
```

Step4: o Access and print the value of the pixel at coordinates (100, 100). o Modify the color of the pixel at (200, 200) to white.

In [34]:
```
# Modify a block of pixels (300x300) to white, starting from (200, 200)
image[200:500, 200:500] = [255, 255, 255]  # Rows: 200-499, Columns: 200-499
```

In [35]:
```
# Convert BGR to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

In [36]:
```
# Display the modified image
plt.imshow(image_rgb)
plt.title("Image with 300x300 White Block")
plt.axis("off")
plt.show()
```



Step5: o Resize the original image to half its size and display it.

In [37]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')
```

In [38]:
```
image.shape
```

Out[38]:
```
(750, 750, 3)
```

In [39]:
```
# Resize the image to half its size
resized_image = cv2.resize(image, (750 // 2, 750 // 2))  # (new_width, new_height)
```

In [40]:
```
# Convert BGR to RGB for displaying with Matplotlib
resized_image_rgb = cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB)
```

In [41]:
```
resized_image_rgb.shape
```

Out[41]:
```
(375, 375, 3)
```

In [42]:
```
# Display the resized image
plt.imshow(resized_image_rgb)
plt.title("Resized Image of Dolphin (Half Size)")
plt.axis("off")
plt.show()
```

Step6: o Crop a region of interest (ROI) from the image (e.g., a 100x100 pixel area starting at (50, 50)) and display it.

In [43]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')
```

In [44]:
```
image.shape
```

Out[44]:
```
(750, 750, 3)
```

In [45]:
```
# Crop a 300x300 region starting from (50, 50)
roi = image[50:350, 50:350]  # Rows: 50-349, Columns: 50-349
```

In [46]:
```
# Convert BGR to RGB for displaying with Matplotlib
roi_rgb = cv2.cvtColor(roi, cv2.COLOR_BGR2RGB)
```

In [47]:
```
# Display the cropped region (ROI)
plt.imshow(roi_rgb)
plt.title("Cropped Region of Interest (ROI) image of Dolphin")
plt.axis("off")
plt.show()
```

Step7: o Flip the original image horizontally and display it. o Flip the original image vertically and display it.

In [48]:
```
# Load the image
image = cv2.imread('dolphin image 1.jpg')
```

In [49]:
```
# Flip the image horizontally (left-right)
flipped_horizontally = cv2.flip(image, 1)
```

In [50]:
```
# Convert BGR to RGB for displaying with Matplotlib
flipped_horizontally_rgb = cv2.cvtColor(flipped_horizontally, cv2.COLOR_BGR2RGB)
```

In [51]:
```
# Horizontal flip
plt.imshow(flipped_horizontally_rgb)
plt.title("Flipped Horizontally Dolphin")
plt.axis("off")
```

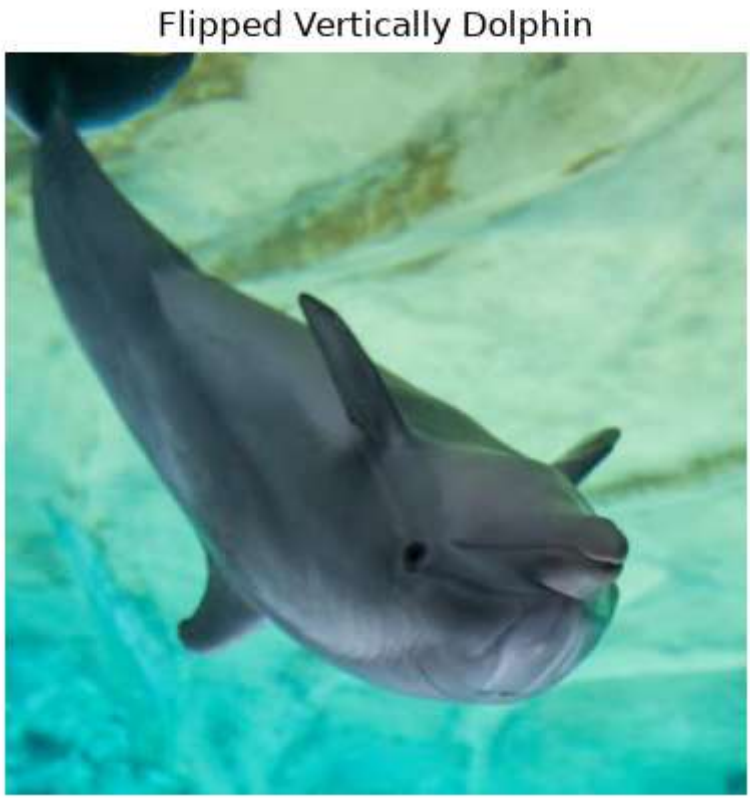
Out[51]:
```
(np.float64(-0.5), np.float64(749.5), np.float64(749.5), np.float64(-0.5))
```

In [52]:
```
# Flip the image vertically (up-down)
flipped_vertically = cv2.flip(image, 0)
```

In [53]:
```
# Convert BGR to RGB for displaying with Matplotlib
flipped_vertically_rgb = cv2.cvtColor(flipped_vertically, cv2.COLOR_BGR2RGB)
```

In [54]:
```
# Vertical flip
plt.imshow(flipped_vertically_rgb)
plt.title("Flipped Vertically Dolphin")
plt.axis("off")
```

Out[54]:
```
(np.float64(-0.5), np.float64(749.5), np.float64(749.5), np.float64(-0.5))
```



Step8: o Save the final modified image to your local directory.